Lab 00 — The Linux fundamentals Docker assumes you know

Theory — the kernel, processes, users, files, the shell and three networking notions. Everything the following labs will use without ever explaining it again.

Objectives

- Place the three floors of a Linux system: kernel, system calls, userland.
- Describe a process: PID, parent, environment, signals, exit code.
- Read a line of `ls -l`: owner, group, permissions — and know what `root` can do that you cannot.
- Use the shell as a tool: environment variables, `PATH`, redirections, pipes.
- Connect each notion to what it will become in the Docker labs.

---

1. Why a Linux lab in a Docker course

Because a container is **nothing but Linux**. When you read, in lab 01, that "a container is an isolated process", the sentence will only serve you if *process* is a precise idea in your mind: something with a number, a parent, an environment, a way of dying. Likewise, Podman's "rootless" is incomprehensible without UIDs, a port that is "already taken" without the notion of port, a volume without the notion of mount. This lab installs that vocabulary — only that — and each section announces the lab where the notion will return.

> **→ HISTORY**
> Unix was born in 1969 at Bell Labs; it imposed two ideas that still govern everything: *everything is a file* and *small programs you assemble*. In 1991, a Finnish student, Linus Torvalds, wrote a free Unix-compatible kernel: Linux. Ubuntu (2004) is a **distribution** of it: the Linux kernel plus a chosen, packaged userland. And since 2019, Microsoft has shipped its own Linux kernel inside Windows: that is WSL 2, your lab machine.

2. The kernel and the userland

A Linux system has two floors. Below, the **kernel**: the only program that touches the hardware. It creates processes, distributes CPU time and memory, reads disks, sends network packets. Above, the **userland**: everything else — `bash`, `ls`, `java`, your application. Between the two, a single frontier: the **system calls** (*syscalls*). A program never reads a file by itself; it asks the kernel for `open` then `read`, and the kernel decides whether to allow it.

This frontier explains two things you will see constantly. First, portability: a Linux binary runs on any distribution, because it only asks for system calls, identical everywhere. Second, control: since *everything* goes through the kernel, it is enough for the kernel to lie a little ("you are process 1", "here is your `/`") to isolate a process — which is exactly what a container will do in lab 01.

→ WINDOWS / WSL
A Linux program only "talks" to a Linux kernel; Windows has its own, incompatible one. **WSL 2** (*Windows Subsystem for Linux*) solves the problem by running a real Linux kernel inside a tiny VM managed by Windows. Your Ubuntu 24.04 is a distribution *inside* that VM: there, `uname -r` answers `...microsoft-standard-WSL2`, the signature of the kernel Microsoft compiles. The Windows disk is visible under `/mnt/c`.

3. Processes

A **process** is a program in execution: code, memory, and an identity. The kernel gives it a unique **PID** (*process ID*) and remembers its parent, the **PPID**. Every process is born from another — when you type `ls`, your shell duplicates itself (`fork`) and the clone replaces itself with `ls` (`exec`). At boot, the kernel launches a first process, **PID 1** (`systemd` on Ubuntu), ancestor of all the others; when a parent dies before its child, the orphan is adopted by PID 1. Remember that number: in a container, *your application* will be PID 1, with unexpected responsibilities (lab 03).

→ LINUX
A **daemon** is a service process: launched at boot by `systemd`, detached from any terminal, it runs in the background waiting to be needed — `sshd` waits for SSH connections, `cron` for the time of its tasks. By convention, its name ends in a "d". Remember the word: Docker relies entirely on a daemon, `dockerd`, and Podman defines itself by the absence of one — THE debate of lab 01.

A process always ends by returning an **exit code**: `0` means "success", everything else is a failure. The shell stores it in `$?`. A few conventional values: `1` general error, `2` misuse, `126` file not executable, `127` command not found, `128 + n` killed by signal *n*.

Because you don't "close" a process: you send it a **signal**, a numbered notification from the kernel. The three to know:

Signal	Number	Meaning	Can the process ignore it?
`SIGTERM`	15	"Terminate cleanly"	Yes — it may first save, close, tidy up
`SIGKILL`	9	Immediate death, by the kernel	**No** — and nothing gets tidied
`SIGINT`	2	Keyboard interrupt (`Ctrl+C`)	Yes

REMEMBER
`kill` does not mean "kill" but "send a signal"; by default it sends the polite `SIGTERM`. A process killed by `SIGKILL` exits with code `137` (128 + 9). You will meet that number for the rest of your Docker life: it is the signature of a container stopped by force — often for lack of memory.

The kernel exposes the state of every process in `/proc`, a fake directory: `/proc/1234/` describes process 1234 (its command, its environment, its limits), fabricated on the fly, occupying not one byte of disk. `ps` does nothing but read it.

4. Users, groups, permissions

Every process runs *as* someone: a **user**, identified by a number, the **UID**, and **groups** (GID). The kernel only knows numbers; the names (`kevin`, `postgres`) come from the file `/etc/passwd`. Your first Ubuntu user has UID **1000**. The user `root`, UID **0**, is special: the kernel refuses it nothing. The `sudo` command runs a command *as* root, logging who asked.

Every file has an owner, a group, and nine permission bits, readable in `ls -l`:

```
-rw-r----- 1 root shadow 1234 ... /etc/shadow
 └┬┘└┬┘└┬┘    └──┬──┘ owner root, group shadow
  │  │  └ others: nothing
  │  └ group shadow: read
  └ root: read + write
```

`r` read, `w` write, `x` execute (for a directory: enter it). `chmod` changes these bits, `chown` the owner. A detail that traps everyone: a script must be **executable** (`chmod +x`) to be launched with `./script.sh` — otherwise the shell answers `Permission denied`, code 126.

→ SECURITY
The golden rule: you never work as root, you elevate your rights punctually with `sudo`. It is the Linux version of the principle of least privilege, and the central argument of Podman **rootless**: your containers will run under UID 1000, not UID 0, and a compromised application will only have your rights (lab 01).

PITFALL
The kernel compares **numbers**, not names. A file created by UID 1000 inside a container belongs to UID 1000 everywhere, even if the displayed name changes from one system to another. This obvious fact will become the classic volume headache of lab 06.

5. Files, tree, mounts

Under Unix, *everything is a file*: documents, but also disks (`/dev/sdc`), the kernel's state (`/proc`), sockets. There are no `C:` or `D:` drives: one single tree, starting at the root `/`, where each disk or filesystem is **mounted** — hooked onto a directory. `findmnt /` tells you which disk provides the root; on WSL, `/mnt/c` is the mount of the Windows disk. Mounting, unmounting, stacking filesystems: that is the exact mechanics of Docker images and volumes (labs 02 and 06).

The standard directories to recognize: `/etc` (configuration), `/home` (your files), `/usr/bin` (the programs), `/var` (living data: logs, databases), `/tmp` (temporary), `/proc` and `/sys` (windows onto the kernel).

6. The shell: the environment, the PATH, the plumbing

The **shell** (`bash`) is a process like any other, whose job is to launch the others. Three of its mechanisms are pure "Docker knowledge".

Environment variables. Every process is born with a key=value dictionary inherited from its parent: `HOME`, `PATH`, `LANG`... A shell variable (`MSG=hello`) stays local; it only enters the children's environment after `export MSG`. This is THE configuration channel of containers: in lab 08, your Spring Boot application will read its database password from a variable, never from a file inside the image.

→ JAVA
A JVM is an ordinary process: `java -jar app.jar` has a PID, a UID, variables. Spring Boot reads the environment at startup: `SERVER_PORT=9090` is enough to change its port, without touching the JAR. `System.getenv("HOME")` in Java is a read of that same inherited dictionary.

The `PATH`. When you type `ls`, the shell looks for an executable named `ls` in the list of directories of the `PATH` variable, in order. `which ls` shows what it found; `command not found` (code 127) means "in none of those directories". This is why a script in the current directory is launched as `./script.sh`: "here" is not in the `PATH`, out of caution.

Redirections and pipes. A process has three streams: input (0, *stdin*), output (1, *stdout*) and errors (2, *stderr*). The shell plugs them wherever you want: `> file` diverts the output, `2>` the errors, `2>&1` merges the two, and `command1 | command2` plugs the output of one into the input of the other. You will assemble these pipes in every lab (`podman ps | grep …`), and a container's logs are nothing but its streams 1 and 2, captured (lab 03).

7. Networking in three notions

You need three ideas to survive until lab 07. **The interface**: a machine's network socket, with an IP address; `lo`, the *loopback* interface, carries the address `127.0.0.1`, alias `localhost` — the machine talking to itself. **The port**: a number from 1 to 65535 that distinguishes the services of a single address; only one process listens on a given port, `ss -tlnp` lists who listens where. **The privilege**: ports below 1024 are reserved for root — the reason your test server will listen on 8080 rather than 80, and why Podman rootless will refuse `-p 80:80` (lab 07).

> **→ NETWORK**
> `curl` is the Swiss army knife: it makes an HTTP request and prints the raw response. `curl -i http://localhost:8080/` shows the code (`200 OK`, `404` …), the headers, the body. It is tool number one for testing a containerized API without a browser.

> **→ WINDOWS / WSL**
> WSL 2 automatically relays `localhost`: a server listening on port 8080 *inside* Ubuntu is reachable from a **Windows** browser at `http://localhost:8080`. Convenient — but remember this relay is a favor from WSL, not a property of Linux.

8. In the workplace

The whole container ecosystem is the industrialization of these notions. A production Spring Boot server is: a `java` process (PID) launched by an unprivileged application user (UID), configured through environment variables, writing its logs to *stdout*, listening on port 8080, stopped by `SIGTERM` during deployments. The operator diagnosing an incident chains `ps`, `ss`, `curl`, reads `$?`, and digs through logs with `grep`. Docker will replace none of this: it packages it.

> **PODMAN**
> Podman pushes this logic all the way: no daemon, just *your* user (UID 1000) launching processes. This entire lab — UIDs, signals, `/proc`, unprivileged ports — is the exact description of what Podman is allowed to do without `sudo`. Docker, by contrast, relies on a daemon running as root (`dockerd`); that difference will fill lab 01.

Remember

- The **kernel** controls everything; programs only make **system calls**. Isolating a process means making the kernel lie — the founding idea of the container.
- A **process** has a PID, a parent, an inherited environment, and ends with an **exit code**: `0` = success, `137` = killed by SIGKILL.

- `SIGTERM` asks politely, `SIGKILL` executes without appeal. A well-behaved service stops on SIGTERM.
- The kernel reasons in numeric **UID/GID**; `root` = UID 0 = every right; `sudo` elevates punctually.
- One single file tree; disks and virtual filesystems are **mounted** into it; `/proc` is the window onto the kernel.
- **Environment variables** pass from parent to child; the `PATH` decides which commands exist.
- A service = an address + a **port**; `localhost` = the machine itself; ports < 1024 reserved for root.

Vocabulary

kernel: the program that controls hardware and processes. — **userland**: everything running above the kernel. — **system call**: a program's request to the kernel (`open`, `fork`…). — **process**: a program in execution, identified by a **PID**. — **PID 1**: first process, ancestor and guardian of all. — **daemon**: background service process, without a terminal, managed by `systemd` (`sshd`, `dockerd`). — **signal**: notification sent to a process (`SIGTERM`, `SIGKILL`). — **exit code**: integer returned at a process's death, `0` = success, stored in `$?`. — **UID / GID**: user and group numbers, the only thing the kernel understands. — **root**: UID 0, no check applies. — **mount**: attaching a filesystem to a directory of the tree. — **/proc**: virtual tree exposing the state of the kernel and of processes. — **environment variable**: key=value pair inherited by child processes. — **PATH**: list of directories where the shell looks for commands. — **stdin / stdout / stderr**: the three standard streams (0, 1, 2). — **pipe**: plugging one process's output into another's input. — **port**: number identifying a service on an IP address. — **localhost**: `127.0.0.1`, the machine's own address.